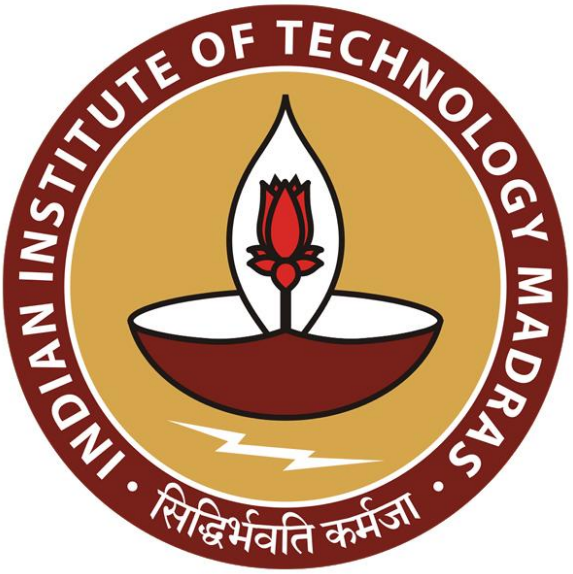
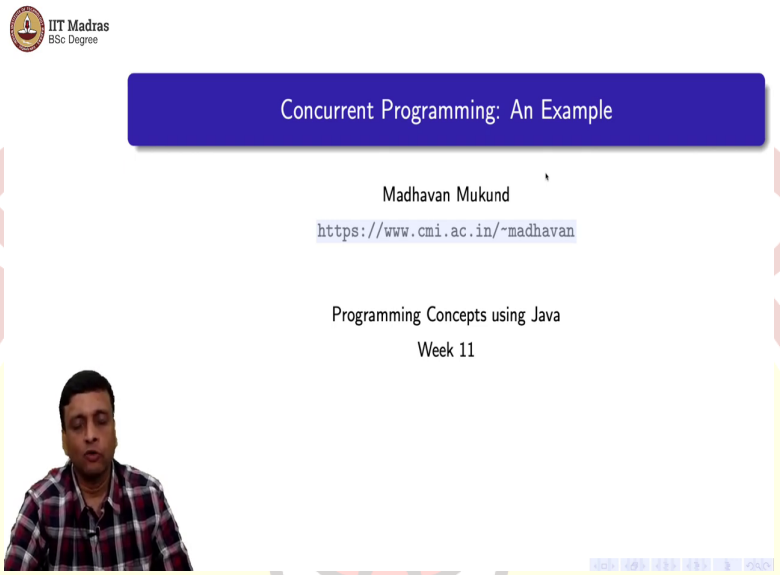


IIT Madras

ONLINE DEGREE

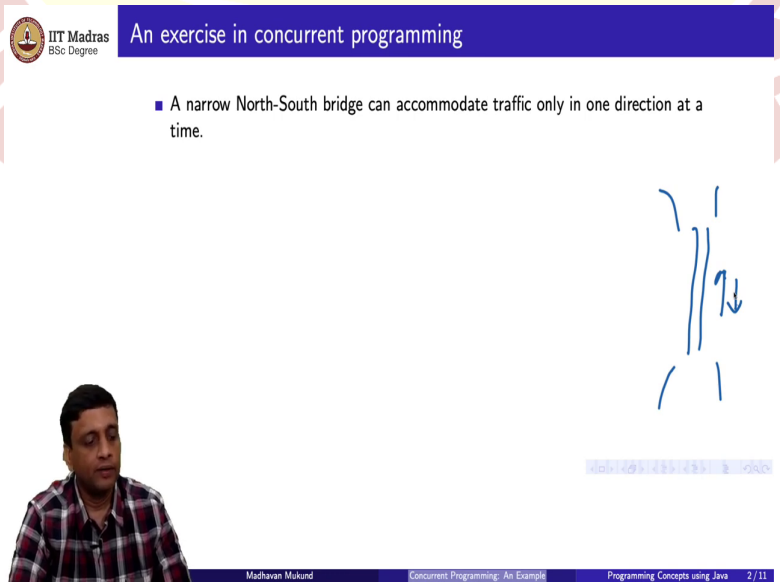
Programming Concepts using Java
Professor Madhavan Mukund
Department of Computer Science
Chennai Mathematical Institute
Indian Institute of Technology Madras
Concurrent Programming: An Example

(Refer Slide Time: 00:13)



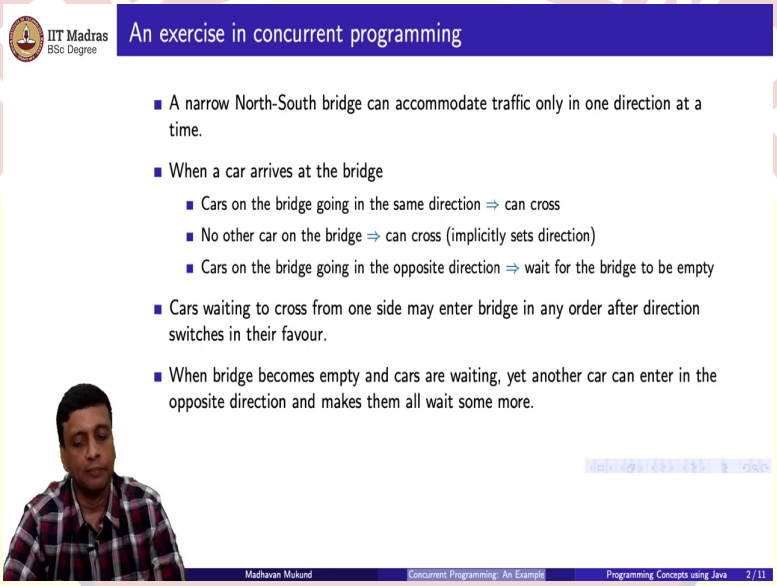
So, let us look at an example of how you would program some object which occurs, which allows some concurrent access using diverse synchronization primitives. So, this is a toy example. But it is illustrative of the many of the kinds of things that you would like to allow in a concurrent situation.

(Refer Slide Time: 00:31)



So, here is the scenario. So, you have a bridge. So, as you could imagine, there are situations where a bridge is under repair. So, you have traffic coming normally. But at any given time, this bridge is regulated, so that it only allows traffic in one direction. So, you have a north-south bridge, which is currently one way, but it is one way in both ways. So, of course, in a real-life situation, you might have a traffic light or a human being at both ends, who is actually regulating the traffic. But now we have an automatic way to do it. And this is what we want.

(Refer Slide Time: 01:08)



So, when a car arrives at the bridge, the bridges currently either going from south to north, or from north to south. So, if the bridge is currently oriented in the same direction that you want to go, so the cars on the bridge are going in the same direction as the car that arrived in the bridge, then this car can join and cross. Now, if the bridge is empty, then there are two situations that can happen. So, there is no car on the bridge.

But the bridge currently is marked as going in the same direction, of course, you can join. But if the bridge is empty, and the bridge is currently pointing in the opposite direction, then this car can reverse the direction and start a new flow in the opposite direction. So, if there is no other car on the bridge, and a car arrives, then regardless of what the current orientation of the bridges, this car will cross, and once it starts crossing, it will set the orientation of the bridge to whatever its orientation, if it is going south to north the bridge become south to north and vice versa.

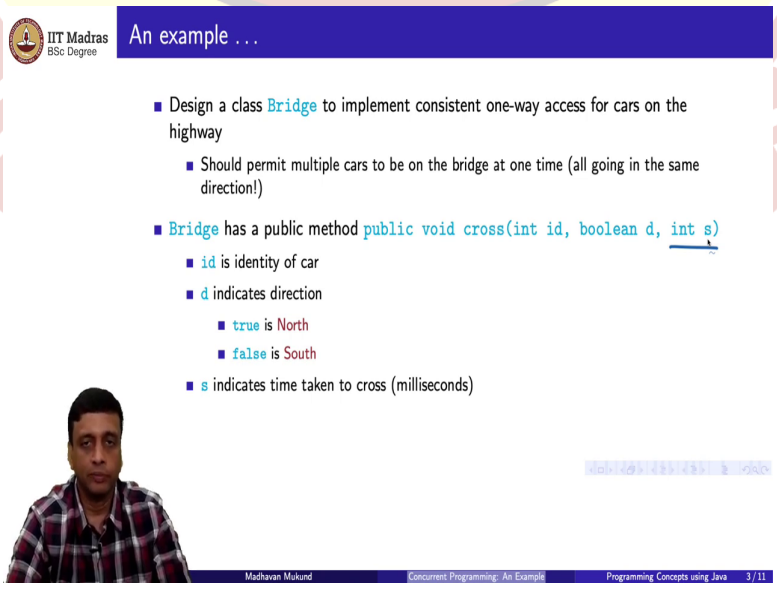
And now, while the bridge is going in one direction, the cars which are have arrived from the opposite, notice that the cars must arrive from the other side of the bridge. So, they are either at the south end of the bridge waiting to go north, or they are at the north end of the bridge waiting to come south, so there is a queue, in the normal sense a traffic jam of cars waiting to get onto the bridge.

And what are they waiting for? Well, they are waiting for the bridge to become available. And the only way that the bridge will become available to them, of course, is for them to get the bridge empty so that they can switch the direction. Otherwise, so long as things keep coming, they are going to be stuck. So, the cars can enter in any order. So, it is, as I said, it is a traffic jam is not really a queue.

So, we are not really going to insist that the cars which are waiting, must enter in the order in which they came to wait. So, once the bridge switches direction, all the cars that are waiting to get onto the bridge can get on they can kind of jockey for space and get on but we do not care in what order they get.

But the other thing that could happen is that you could get unlucky, so you are waiting for the bridge to get empty, the bridge becomes empty. But while you are thinking about switching and getting onto the thing, another car comes and keeps the bridge in the direction that is against it. So, all these things can happen.

(Refer Slide Time: 03:38)



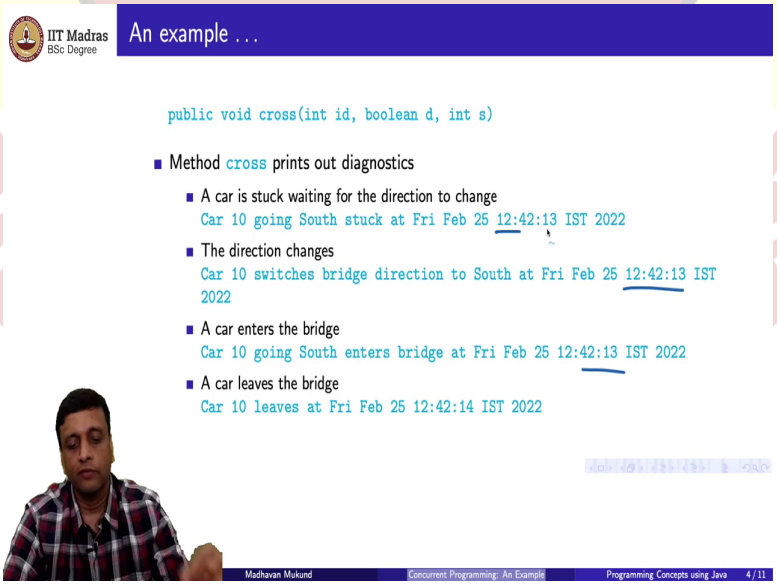
So, what we would like to now do as our concurrent programming exercise is to implement this bridge in Java. So, we want to implement a data structure, which captures this bridge. So, that there is consistent one-way access for cars on the highway. And one of the things that we require is that more than one car can be on the bridge at the same time.

So, it must be something which is shared by multiple cars. So, we will have a car class and we are have a bridge class and each car could be on the bridge and more than one car can be on the bridge at the same time. So, what we will have for the bridge is a method which says a car will say I want to cross the bridge. So, the car will provide its id. So, each car will be numbered, it will provide the direction in which it wants to go.

So, the direction will be either north or south. So, we signified with a Boolean. So, true means north, false mean south, it could be the other way around, but just an arbitrary choice. And finally, in order for this to make sense, each car has to spend some time on the bridge. I mean, if these are instantaneous things then there is not much of a problem.

So, because a car is spending some time on the bridge, then other cars have to wait. So, each car when it wants to cross the bridge it says this is my identity, this is the direction I want to go. And this is how much time I am going to spend when I am on the bridge.

(Refer Slide Time: 05:08)

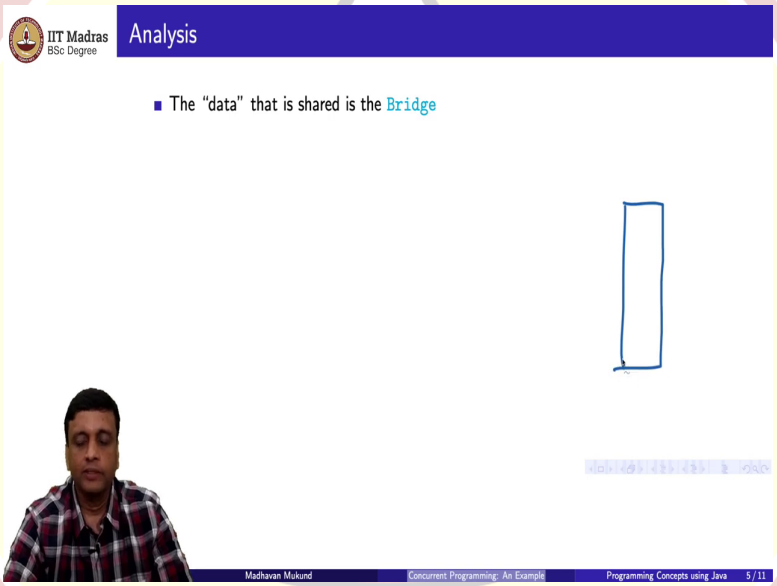


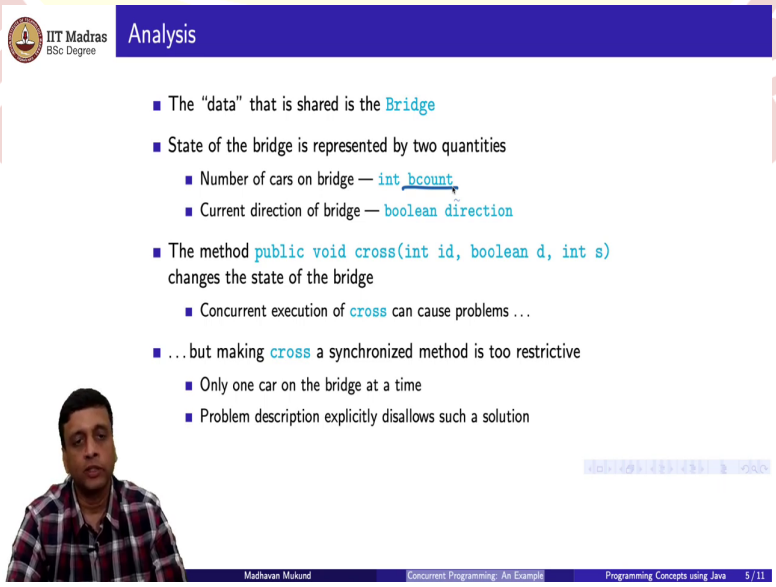
And this method that we are going to define this cross method will print out statistics, some information. For instance, if a car comes and it is has to wait, because the bridge is in the

wrong direction, then it will print out its id, it will say which direction it is going. And it will say this is I mean, just to indicate, so that we know what is happening, what time it happened.

If a car which is waiting, is the first car to get on the bridge, that means that the direction has changed. So, we will attribute it to this car, we will say that this particular car switch the direction from north to south, to south at this time. And then once it is switched the direction, remember, this is not an atomic thing in that sense, you switch the direction, then you get on. So, then we can say that the car enters the bridge. And finally, the car leaves the bridge. So, with everything, there is a timestamp. And you can see that I mean, these timestamps are all identical, but they could be a few milliseconds apart.

(Refer Slide Time: 06:09)





So, how do we approach this problem? So, there is a shared data structure, which is the bridge. So, the bridge is what is shared by all these cars. So, the state of the bridge consists of two things in what direction is it going, and how many cars are on the bridge, that is what I need to know. It is not really relevant to me which cars are there, whether it is car number 17, or car number 20, or whatever, but I need to know how many cars are there, because only when the car is, car count is 0, am I able to switch the direction.

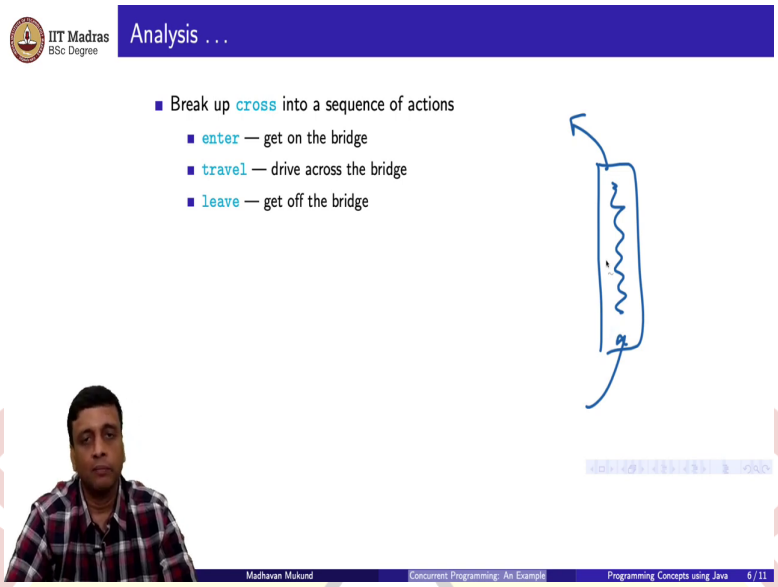
So, I have this bridge count, which I call bcount. And I have this direction, which is either true for north or false for south indicating the direction in which the bridge is currently oriented. So, when I now cross, when a car comes and crosses, it can change the state of the bridge in two ways, it could add one more car which is going in the current direction, in which case the count will change.

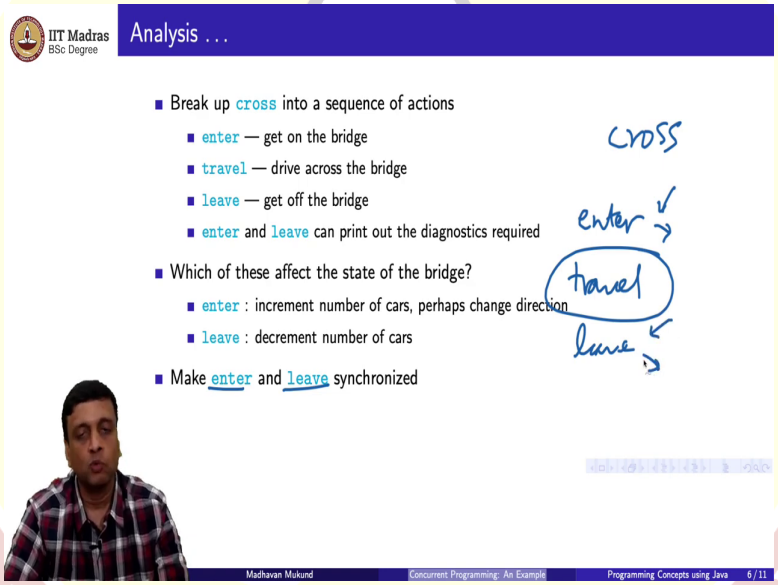
But it could also be the count is 0, in which case it can change the count by making 0 into 1 and switch the direction. So, now we have a shared structure, we have a structure which has these two instance variables bcount and direction. And that is shared by the cars and each car is trying to update it. So, this is very similar to all the other problems we saw with bank accounts and all that, we have multiple threads, each car is going to be a parallel thread. And they are all trying to update this in their favour.

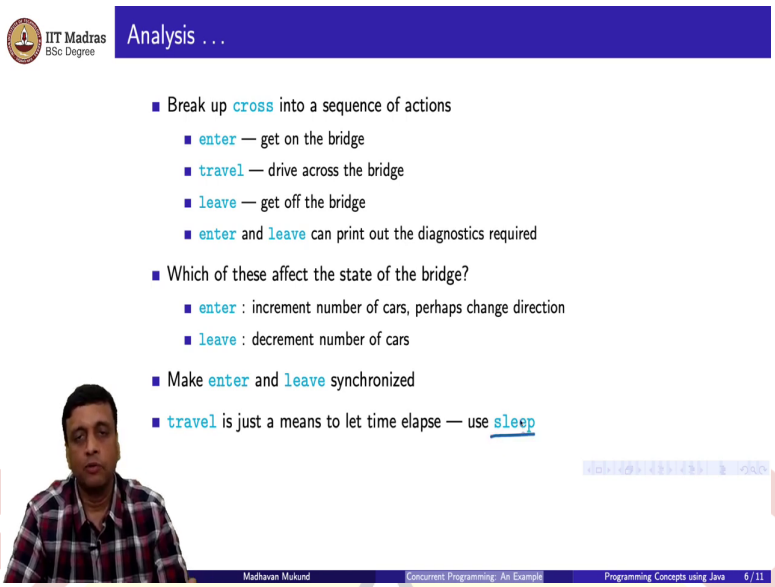
And therefore, if we have a concurrent access to this cross function, then we are going to have possible inconsistencies in the state of the bridge. So, the natural thing to do would be to make cross a synchronized method, to say that no two threads can access cross at the same time. But if we do that, it means that only one car can cross at a time. So, till that car is finished crossing, the cross method will be occupied by that thread and no other car can cross. And this is not what the specification said.

It said that we must allow or we should allow multiple cars to be crossing the same direction at the same time. So, the bridge count, this bcount is not 0 or 1 it is any number 0 or any positive number. So, the problem description requires us to allow that, therefore a sink, it would be a safe solution to this call this cross method synchronized in the Java monitor sense. And make sure that no two cars can do it. But that is not an acceptable solution given what we want.

(Refer Slide Time: 08:41)







So, to understand what is happening, we can think of cross as a three-step thing. So, I entered the bridge, then I spent some amount of time going there, which I tell it because I know how much time, part of the cross thing is to say, this is my id, this is the direction I want to go. And this is how many milliseconds I am going to take to cross the bridge. And then finally I leave it.

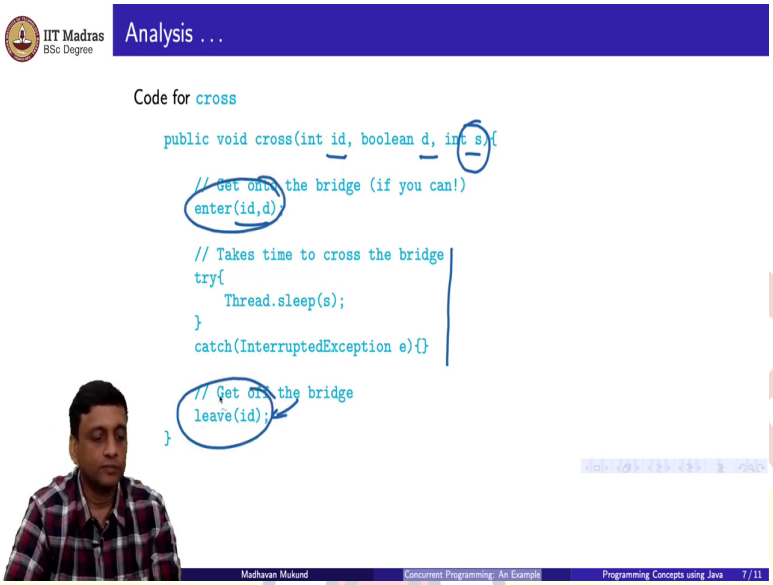
Now, the process of going across the bridge is harmless, but enter and leave are the ones which actually do something. So, that is where either you have to switch the direction or you have to wait or you have to enter and you have to leave. So, all the diagnostics will be printed by these two functions internally. And those are also the functions which affect the state of the bridge. So, when I enter, I will certainly add one car to the bridge.

Remember, enter means I actually get onto the bridge. When I enter onto the bridge, I actually add one car to the bridge and if it was 0 and make it 1, I might change the direction. So, enter could affect and of course leave will decrement the, leave will not change the direction because it does not have any interest in changing direction but it will certainly reduce the number of cars by part.

So, what we really need to do is make sure that enter and leave, so we have a cross function, which consists of three parts, enter, travel, and then leave. So, we want to allow concurrency in the center, we want to allow multiple things to be traveling at a time. But we want to make sure that this enter and leave happen only one at a time. So, that is what we want to make

synchronized. And travel, we just want to kill time. So, we will just use sleep to simulate travel.

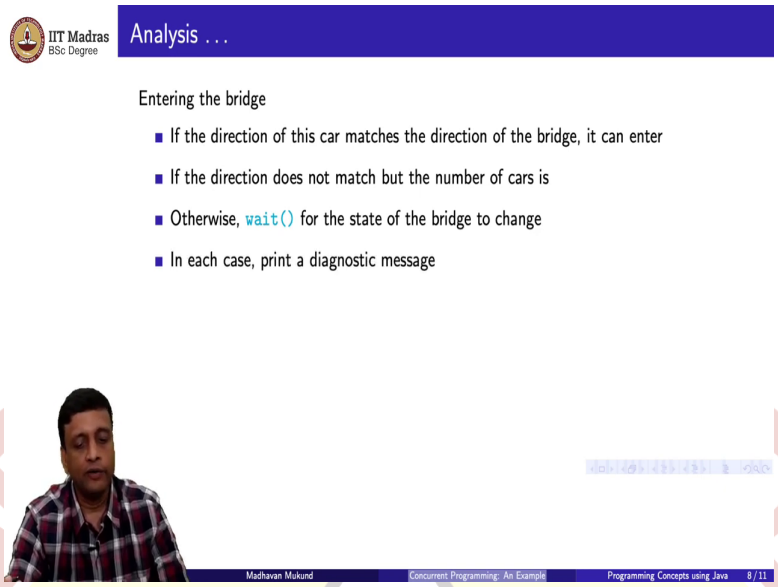
(Refer Slide Time: 10:29)

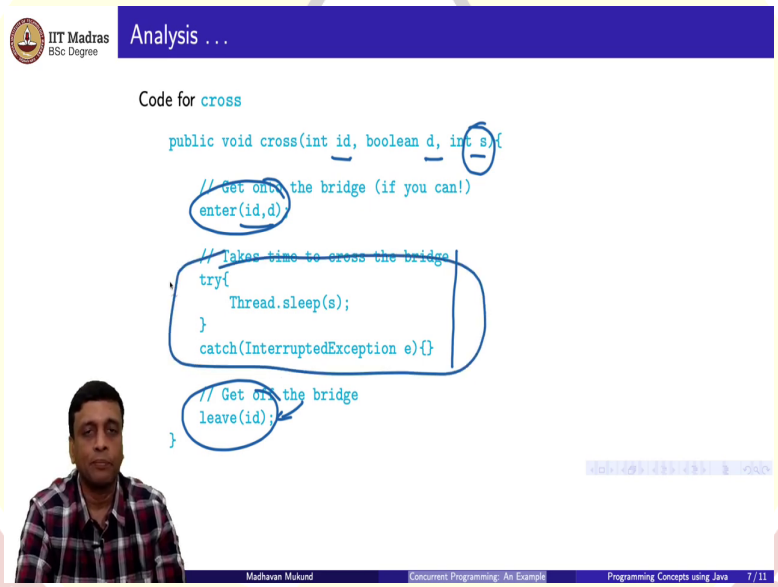


So, in other words, this is what cross will look like. We will first, so cross has three parts, the id of the car, the direction and how much time, so we will first say I want to enter the bridge. This is my id, and this is a direction. Now, we may get stuck at this point, because this is going to be a synchronous method. Then having gotten this car, this thread is going to just sleep for this much time.

And remember, sleep throws this interrupted exception. So, when we say thread dot sleep, we have to catch that exception, or we have to promote that exception, and so on. And finally, at the end of this sleep, we are going to try and leave the bridge. But again, leaving the bridge is going to be subject to some mutual exclusion, we do not want two cars to leave at the same time and update the decrement that counter in an inconsistent way. So, these two are going to be synchronized for us.

(Refer Slide Time: 11:22)

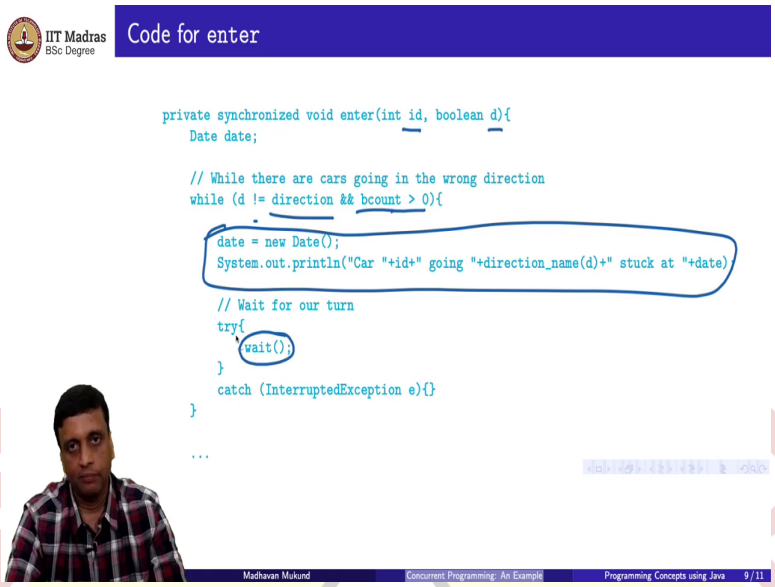




So, let us look at entering the bridge now. So, travel is basically just been captured by this one try thing, so we do not need a separate thing for travel. Travel is a very easy thing. So, we need enter and leave. So, if the direction of this car is equal to the direction of the bridge, you can enter, if it does not match, but the number of cars is greater than 0 is sorry, if the direction does not match, but the number of cars is 0, is greater than 0, then you must wait.

So, either you get there and you are going in the same direction that the bridge is oriented correctly, you can get on or you are going in the opposite direction, the bridge is not empty you must wait.

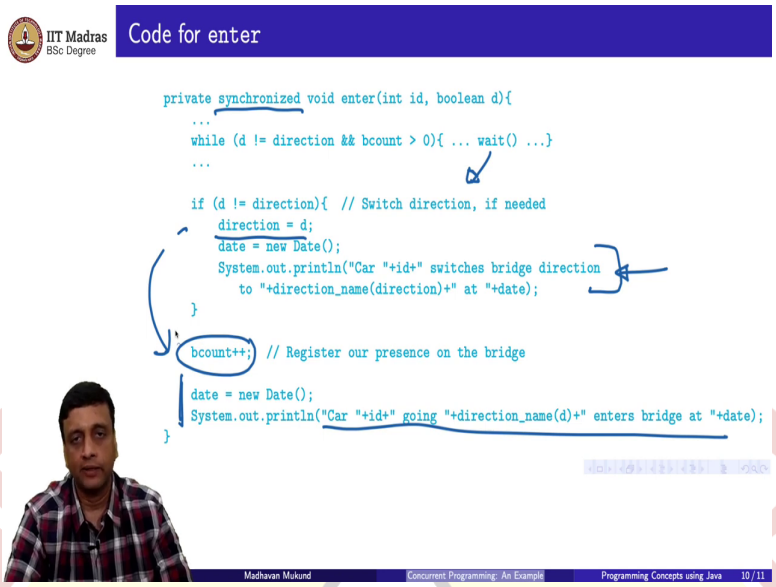
(Refer Slide Time: 12:14)



And when you wait for this thing, you must print a diagnostic message. So, this is what it says. So, I enter with an id and a direction. And remember, this bridge has this bcount and the direction. So, if my direction is not equal to the direction of the bridge, and simultaneously there are cars in the bridge, so the bridges in the opposite direction and the cars on the bridge because bcount is greater than 0 then I must wait.

So, I print out this message, I get the current date and I print out a message, and then I wait. What am I waiting for? I am waiting for something interesting to happen, either the bridge count must come to 0 or some other car must set the direction forward. So, this is the first part of enter that is I check whether I am able to get on the bridge. If I am able to get on the bridge, that is if the direction is then I can get on. Or if the bridge count is 0, I am going to be able to get passes and get on as we will see. But if both of these are not the case, then I must wait at this point for somebody to notify me.

(Refer Slide Time: 13:14)



So, what happens after this I have got notified or I have passed this condition. So, I come here. So, at this point, remember that this is a synchronized method. At this point, this thread is the only thread which has access to bridge. So, when I have come here, I am very sure, that the either the direction is in my favour, or the bridge count to 0. So, the direction is not in my favour, it must be that the bridge count is 0.

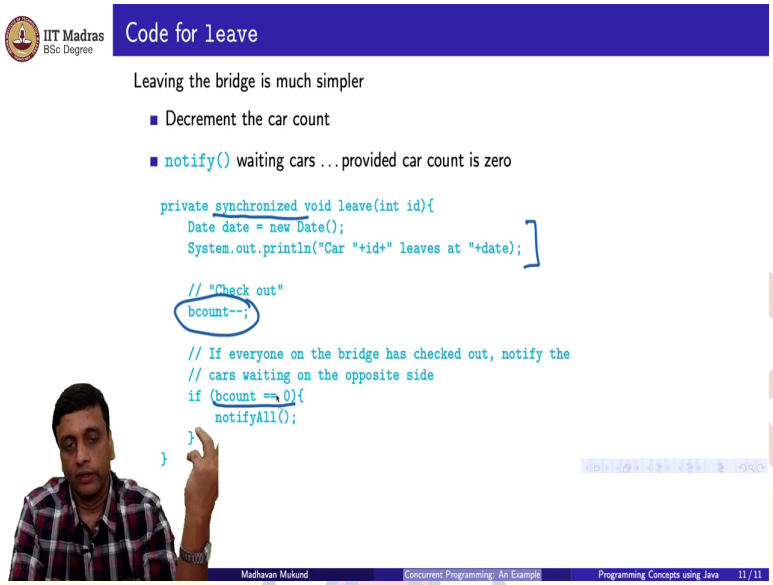
So, I can safely set the direction to my direction, then I print out a message. And now what I do is, so if the direction was not wrong, then I do not print out a message because that means somebody else already switched it. Then I increment the number of cars on the bridge to account for myself. And then I say I have entered. So, every car that enters will print out this line, only the car that switches the direction print out this line. So, now this is a three-step process.

First to check if you can enter. If you cannot enter, you print a message saying I am stuck and you wait, wait means you take the condition variable and you wait. When you are notified, you come out. Either you have got the right direction or the bridge count is 0. If you got the right direction, you just skip pass this and go ahead. If this is the wrong direction, then you first reset the direction to be your direction mentioned that in diagnostic, then.

So, the diagnostic is just so that externally we can observe this thing run, that is the only reason we have this diagnostic. It is not part of the requirement actually, although we did not say it, but it is just to make sure that there is something sensible is happening. And then you

come out and you print it, so that is how enter works. So, remember that we had these two synchronized methods enter and leave so now this completely describes enter.

(Refer Slide Time: 14:57)

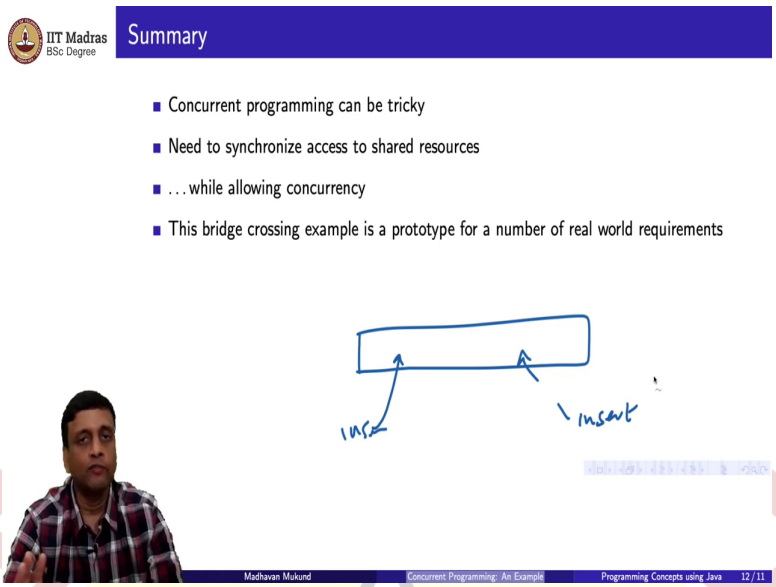


Leaving the bridge does not require so much work because you can always leave the bridge. It is only a question of being synchronized. So, you want to decrement the car count. But you also want to notify cars that may be waiting for the bridge to become empty. But you only notify if your decrement makes it 0, if you decrement and there are still 7 cars in the bridge, there is no point notifying anybody.

So, again, it is a synchronized method. So, when you do, when you get access to leave, when the thread gets to execute leave, it is the only thread which is manipulating bcount and direction. So, direction is irrelevant. But what you do is you print out your message saying I am leaving, then you check out. You get out of the bridge by decrementing.

And now you say, oh, how many cars are left. So, if after I leave, the count is going to be 0, then it is a good time to notify everyone. If after I leave the count is not 0, there is no point notifying. So, that is how leave works.

(Refer Slide Time: 15:56)



So, this example is to illustrate that concurrent programming is kind of, it is kind of interesting, it is like solving a puzzle. So, you get a description, which says, I need some restricted access to this bridge. But I would also like some flexible access in the form of concurrently cars coming and going. So, it does not have to be, everything is synchronized is not a good solution.

So, you need to access synchronize, you need synchronous access to the shared resources. But you also want to allow some concurrency for performance or whatever. So, this is a toy example. But you can imagine that the same thing could be done in a data structure. So, supposing I have a data structure. Like a list, for example, like a linked list. So, I might want to insert something here and insert something there.

Now normally, we have seen that when we have a data structure. If we want to make sure that there is no incorrect update, we will synchronize the data structure and make sure only one update happens at a time. But if these updates are happening far apart at disjoint parts of the data structure, then actually these two modifications will not interfere with each other.

So, this kind of thing we would like to do for performance. So, we have a large data structure. And we would like multiple threads to access this data structure. When they overlap, we want them to be careful. When they do not overlap, we want them to go away. So, all these kinds of programming tasks required this kind of careful coordination. So, this bridge crossing example though it is a very toy puzzle like example, it illustrates the challenges that are there when we actually want to do effective concurrent programming.